Machine Learning Project
Crime Category Classification

1. Ibrahim Hazem El-sayed
2. Ahmed salah El-Sayed
3. Ismail Mohammed Ismail
4. Ahmed Emad Ibrahim
5. Ahmed Walid El-Sayed
6. Ahmed Ashraf Mohammed
7. Ahmed Ossama Abd El-Gawad
8. El-Shahat Ibrahim El-Shahat
9. Ammar Hassan Ahmed

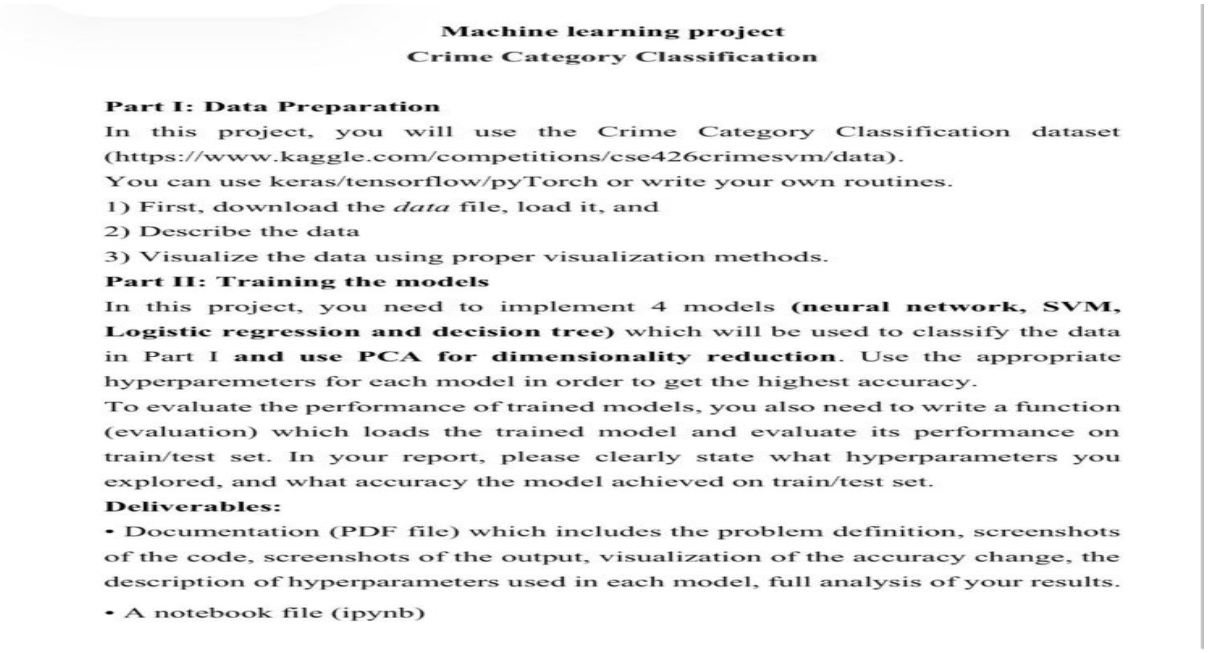
TEAM MEMBERS AND THEIR CONTRIBUTIONS

Team Member	Contributions
Ibrahim Hazem	• Implemented and optimized the **SVM** model. • Implemented and optimized the **Logistic Regression** model.
Ahmed Osama	• Implemented and trained the **CNN (Convolutional Neural Network)** model.
Elshahat Ibrahim	• Implemented and optimized the **Decision Tree** model.
Ahmed Ashraf	• Handled importing and managing all required **libraries** and dependencies.
Ahmed Salah	• Performed the **data loading** and **exploration**. • Conducted **data cleaning** and **preprocessing**. • Feature engineering and **PCA** for dimensionality reduction. • Generated and analyzed the **Confusion Matrix** for all models.
Ammar Hassan & Ahmed Walid	• **Evaluated** model performance using appropriate metrics. • Created **visualizations** (accuracy plots, comparisons, etc.). • Performed **predictions** on the test set. • Final **analysis**, conclusions, and result interpretation. • Prepared the **final outputs** and results presentation.



NOTE:
Ismail Mohamed and **Ahmed Emad** reviewed and refined the entire code, fixed **errors**, and performed **debugging** to ensure the project runs smoothly.

Project Requirements Screenshot



Machine learning project
Crime Category Classification

Part I: Data Preparation
In this project, you will use the Crime Category Classification dataset (https://www.kaggle.com/competitions/cse426crimesvm/data).
You can use keras/tensorflow/pyTorch or write your own routines.
1) First, download the *data* file, load it, and
2) Describe the data
3) Visualize the data using proper visualization methods.

Part II: Training the models
In this project, you need to implement 4 models **(neural network, SVM, Logistic regression and decision tree)** which will be used to classify the data in Part I **and use PCA for dimensionality reduction**. Use the appropriate hyperparemeters for each model in order to get the highest accuracy.
To evaluate the performance of trained models, you also need to write a function (evaluation) which loads the trained model and evaluate its performance on train/test set. In your report, please clearly state what hyperparameters you explored, and what accuracy the model achieved on train/test set.

Deliverables:
- Documentation (PDF file) which includes the problem definition, screenshots of the code, screenshots of the output, visualization of the accuracy change, the description of hyperparameters used in each model, full analysis of your results.
- A notebook file (ipynb)

Requirement Coverage

Requirement	Coverage in Uploaded Notebook
Download/load data file	Satisfied: notebook loads train and test files using pandas.
Describe data	Satisfied: notebook prints columns, info, missing values, distribution and number of categories.
Visualize data	Satisfied: notebook includes category, day-of-week and district bar charts.
Logistic Regression	Satisfied: LogisticRegression model is implemented and evaluated.
SVM	Satisfied: SGDClassifier with hinge loss is used as a scalable linear SVM.
Decision Tree	Satisfied: DecisionTreeClassifier is implemented and evaluated.
Neural Network/CNN	Satisfied: TensorFlow/Keras CNN model is implemented and evaluated.
PCA dimensionality reduction	Satisfied: TruncatedSVD is used as PCA-like dimensionality reduction for sparse TF-IDF features.
Evaluation function	Satisfied: train_and_evaluate_sklearn function trains and evaluates models.
Accuracy comparison and confusion matrix	Satisfied: notebook includes accuracy plots and confusion matrices.

Project Summary and Analysis

Problem Definition: The goal is to classify crime categories based on available information such as time, day, district, address and coordinates. The target column is Category.

Data Leakage Avoidance: The columns Descript and Resolution are removed because they may directly reveal the crime category or post-event information. Removing them makes the evaluation more realistic.

Feature Engineering: The Dates column is converted into Year, Month, Day, Hour, Minute and IsWeekend features. This lets the models learn time-related crime patterns.

Text Processing: The Address column is converted to numerical features using TF-IDF. This allows the models to use address text without treating it as raw strings.

Dimensionality Reduction: TruncatedSVD is used instead of regular PCA because TF-IDF produces sparse matrices. SVD reduces the feature dimension while keeping the pipeline memory-safe.

Models: The notebook implements Logistic Regression, SVM using SGDClassifier with hinge loss, Decision Tree, and CNN using TensorFlow/Keras.

Evaluation: The notebook uses accuracy, classification report, accuracy comparison plots, training time plots and confusion matrix to analyze model performance.

Notebook Cells with Outputs

Cell 1 - Markdown

Crime Category Classification — Final Complete Version

This notebook includes:
- Full data loading
- Data description
- Visualizations
- Preprocessing
- PCA/SVD
- Logistic Regression
- SVM
- Decision Tree
- CNN
- Accuracy comparison plots
- Confusion matrix
- Final submission file

Cell 2 - Code

Code:

```
import os, time, gc, warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.tree import DecisionTreeClassifier

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

print("Imports completed successfully")
```

```
print("TensorFlow:", tf.__version__)
print(os.listdir())
```

Output:

Text output 1:

```
Imports completed successfully
TensorFlow: 2.21.0
['final_complete_crime_project_with_confusion_matrix.ipynb', 'test.csv', 'train.csv']
```

Cell 3 - Code

Code:

```
def find_file(names):
    for name in names:
        if os.path.exists(name):
            return name
    return None

train_path = find_file(["train.csv", "train", "Train.csv", "Train"])
test_path = find_file(["test.csv", "test", "Test.csv", "Test"])

if train_path is None:
    raise FileNotFoundError("Train file not found")
if test_path is None:
    raise FileNotFoundError("Test file not found")

start = time.time()
train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)

print("Train file:", train_path)
print("Test file:", test_path)
print("Loading time minutes:", round((time.time() - start) / 60, 2))
print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)

display(train_df.head())
display(test_df.head())
```

Output:

Text output 1:

```
Train file: train.csv
Test file: test.csv
Loading time minutes: 0.02
Train shape: (878049, 9)
Test shape: (884262, 7)
```

Display output 2:

```
                 Dates        Category                     Descript  \
0  2015-05-13 23:53:00        WARRANTS                WARRANT ARREST
1  2015-05-13 23:53:00  OTHER OFFENSES      TRAFFIC VIOLATION ARREST
2  2015-05-13 23:33:00  OTHER OFFENSES      TRAFFIC VIOLATION ARREST
3  2015-05-13 23:30:00   LARCENY/THEFT  GRAND THEFT FROM LOCKED AUTO
4  2015-05-13 23:30:00   LARCENY/THEFT  GRAND THEFT FROM LOCKED AUTO

   DayOfWeek PdDistrict      Resolution                    Address  \
0  Wednesday   NORTHERN  ARREST, BOOKED         OAK ST / LAGUNA ST
1  Wednesday   NORTHERN  ARREST, BOOKED         OAK ST / LAGUNA ST
2  Wednesday   NORTHERN  ARREST, BOOKED  VANNESS AV / GREENWICH ST
3  Wednesday   NORTHERN            NONE   1500 Block of LOMBARD ST
4  Wednesday       PARK            NONE  100 Block of BRODERICK ST
```

```
           X          Y
0 -122.425892  37.774599
1 -122.425892  37.774599
2 -122.424363  37.800414
3 -122.426995  37.800873
4 -122.438738  37.771541
```

Display output 3:

```
   Id                Dates DayOfWeek PdDistrict                   Address  \
0   0  2015-05-10 23:59:00    Sunday    BAYVIEW   2000 Block of THOMAS AV
1   1  2015-05-10 23:51:00    Sunday    BAYVIEW        3RD ST / REVERE AV
2   2  2015-05-10 23:50:00    Sunday   NORTHERN    2000 Block of GOUGH ST
3   3  2015-05-10 23:45:00    Sunday  INGLESIDE  4700 Block of MISSION ST
4   4  2015-05-10 23:45:00    Sunday  INGLESIDE  4700 Block of MISSION ST

           X          Y
0 -122.399588  37.735051
1 -122.391523  37.732432
2 -122.426002  37.792212
3 -122.437394  37.721412
4 -122.437394  37.721412
```

Cell 4 - Code

Code:

```
print("Columns:")
print(train_df.columns)

print("\nInfo:")
print(train_df.info())

print("\nMissing values:")
print(train_df.isnull().sum())

print("\nCategory distribution:")
print(train_df["Category"].value_counts())

print("\nNumber of categories:", train_df["Category"].nunique())
```

Output:

Text output 1:

```
Columns:
Index(['Dates', 'Category', 'Descript', 'DayOfWeek', 'PdDistrict',
       'Resolution', 'Address', 'X', 'Y'],
      dtype='str')

Info:
<class 'pandas.DataFrame'>
RangeIndex: 878049 entries, 0 to 878048
Data columns (total 9 columns):
 #   Column      Non-Null Count   Dtype
---  ------      --------------   -----
 0   Dates       878049 non-null  str
 1   Category    878049 non-null  str
 2   Descript    878049 non-null  str
 3   DayOfWeek   878049 non-null  str
 4   PdDistrict  878049 non-null  str
 5   Resolution  878049 non-null  str
 6   Address     878049 non-null  str
 7   X           878049 non-null  float64
 8   Y           878049 non-null  float64
dtypes: float64(2), str(7)
memory usage: 60.3 MB
None
```

```
Missing values:
Dates         0
Category      0
Descript      0
DayOfWeek     0
PdDistrict    0
Resolution    0
Address       0
X             0
Y             0
dtype: int64

Category distribution:
Category
LARCENY/THEFT                  174900
OTHER OFFENSES                 126182
NON-CRIMINAL                    92304
ASSAULT                         76876
DRUG/NARCOTIC                   53971
VEHICLE THEFT                   53781
VANDALISM                       44725
WARRANTS                        42214
BURGLARY                        36755
SUSPICIOUS OCC                  31414
MISSING PERSON                  25989
ROBBERY                         23000
FRAUD                           16679
FORGERY/COUNTERFEITING          10609
SECONDARY CODES                  9985
WEAPON LAWS                      8555
PROSTITUTION                     7484
TRESPASS                         7326
STOLEN PROPERTY                  4540
SEX OFFENSES FORCIBLE            4388
DISORDERLY CONDUCT               4320
DRUNKENNESS                      4280
RECOVERED VEHICLE                3138
KIDNAPPING                       2341
DRIVING UNDER THE INFLUENCE      2268
RUNAWAY                          1946
LIQUOR LAWS                      1903
ARSON                            1513
LOITERING                        1225
EMBEZZLEMENT                     1166
SUICIDE                           508
FAMILY OFFENSES                   491
BAD CHECKS                        406
BRIBERY                           289
EXTORTION                         256
SEX OFFENSES NON FORCIBLE         148
GAMBLING                          146
PORNOGRAPHY/OBSCENE MAT            22
TREA                                6
Name: count, dtype: int64

Number of categories: 39
```

Cell 5 - Code

Code:

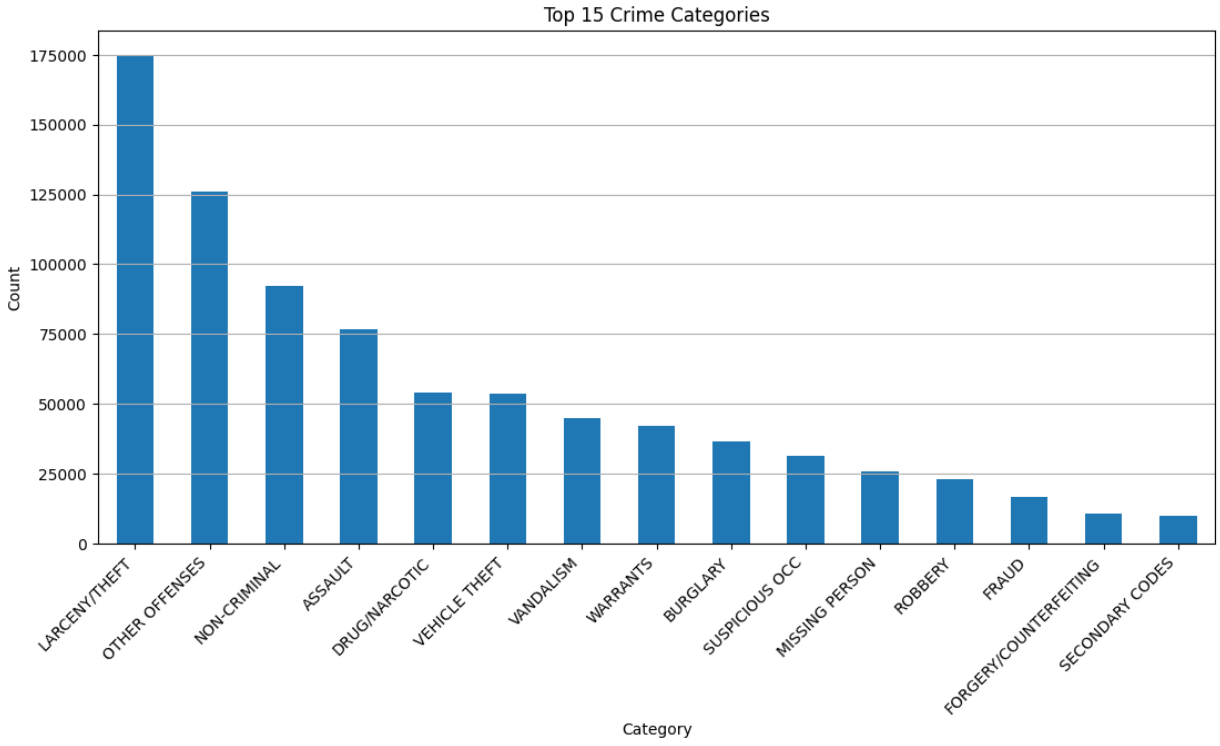
```
plt.figure(figsize=(13, 6))
train_df["Category"].value_counts().head(15).plot(kind="bar")
plt.title("Top 15 Crime Categories")
plt.xlabel("Category")
plt.ylabel("Count")
plt.xticks(rotation=45, ha="right")
plt.grid(axis="y")
plt.show()
```

```
plt.figure(figsize=(10, 5))
train_df["DayOfWeek"].value_counts().plot(kind="bar")
plt.title("Crime Count by Day of Week")
plt.xlabel("Day")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()

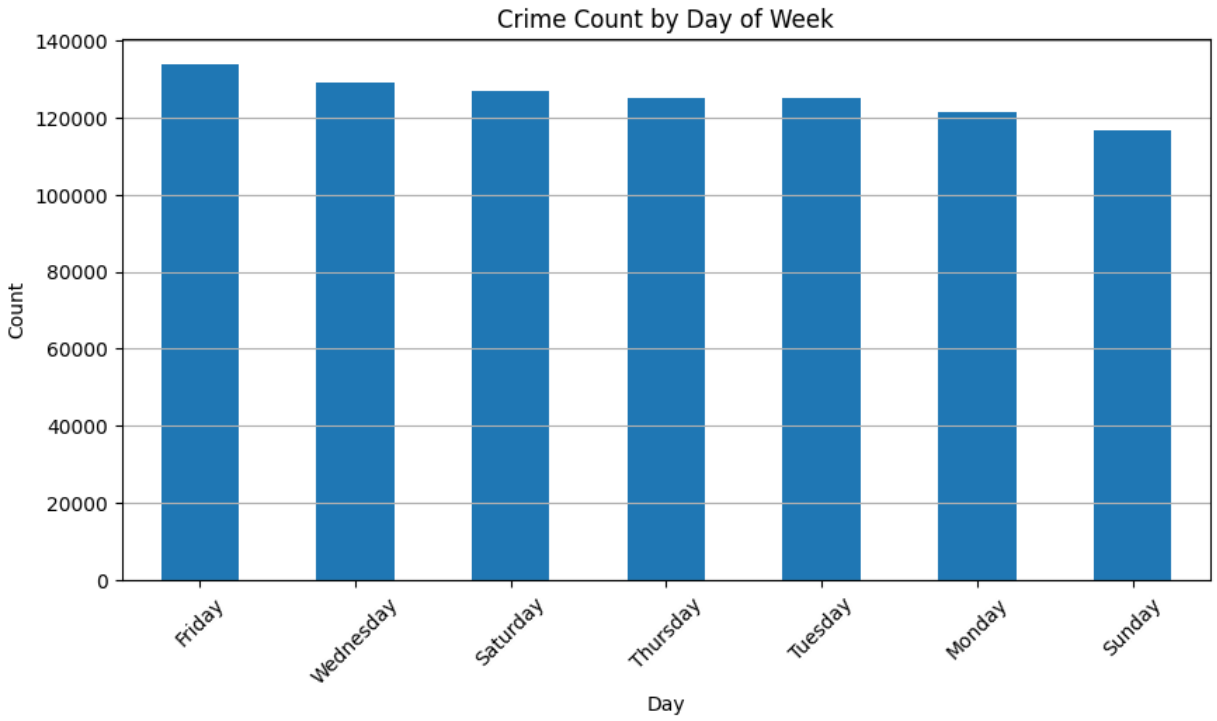
plt.figure(figsize=(10, 5))
train_df["PdDistrict"].value_counts().plot(kind="bar")
plt.title("Crime Count by Police District")
plt.xlabel("Police District")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()
```

Output:

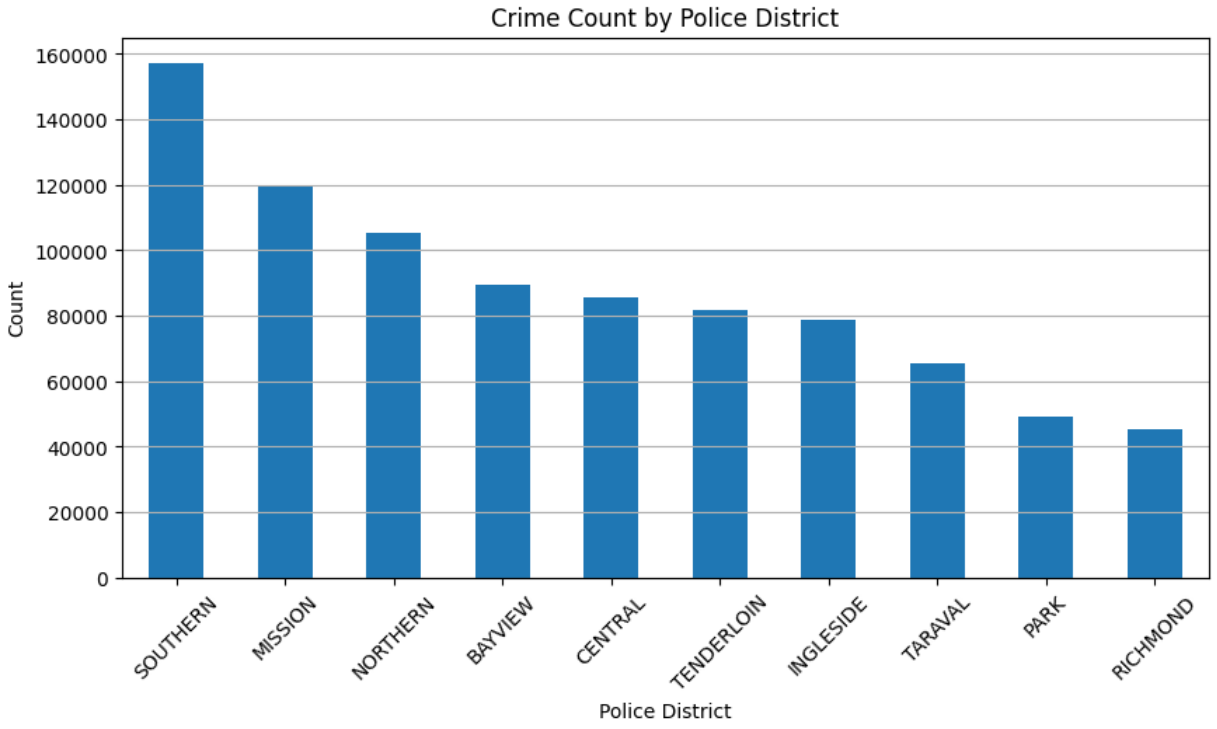
Screenshot output 1:



Screenshot output 2:



Screenshot output 3:



Cell 6 - Code

Code:

```
def prepare_data(df):
    df = df.copy()
    df["Dates"] = pd.to_datetime(df["Dates"], errors="coerce")
    df["Year"] = df["Dates"].dt.year
    df["Month"] = df["Dates"].dt.month
    df["Day"] = df["Dates"].dt.day
    df["Hour"] = df["Dates"].dt.hour
    df["Minute"] = df["Dates"].dt.minute
    df["IsWeekend"] = df["DayOfWeek"].isin(["Saturday", "Sunday"]).astype(int)

    for col in ["DayOfWeek", "PdDistrict", "Address"]:
        df[col] = df[col].fillna("Unknown").astype(str)

    for col in ["X", "Y", "Year", "Month", "Day", "Hour", "Minute", "IsWeekend"]:
        df[col] = df[col].fillna(df[col].median())

    return df

start = time.time()
train_df = prepare_data(train_df)
test_df = prepare_data(test_df)

print("Feature engineering time minutes:", round((time.time() - start) / 60, 2))
```

Output:

Text output 1:

```
Feature engineering time minutes: 0.02
```

Cell 7 - Code

Code:

```
target_col = "Category"

X = train_df.drop(columns=["Category", "Dates", "Descript", "Resolution"], errors="ignore")
y = train_df[target_col]

X_test_final = test_df.drop(columns=["Id", "Dates", "Descript", "Resolution"], errors="ignore")
test_ids = test_df["Id"] if "Id" in test_df.columns else np.arange(len(test_df))

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
num_classes = len(label_encoder.classes_)

print("X shape:", X.shape)
print("X_test_final shape:", X_test_final.shape)
print("Number of classes:", num_classes)
print(label_encoder.classes_)
```

Output:

Text output 1:

```
X shape: (878049, 11)
X_test_final shape: (884262, 11)
Number of classes: 39
['ARSON' 'ASSAULT' 'BAD CHECKS' 'BRIBERY' 'BURGLARY' 'DISORDERLY CONDUCT'
 'DRIVING UNDER THE INFLUENCE' 'DRUG/NARCOTIC' 'DRUNKENNESS'
 'EMBEZZLEMENT' 'EXTORTION' 'FAMILY OFFENSES' 'FORGERY/COUNTERFEITING'
 'FRAUD' 'GAMBLING' 'KIDNAPPING' 'LARCENY/THEFT' 'LIQUOR LAWS' 'LOITERING'
 'MISSING PERSON' 'NON-CRIMINAL' 'OTHER OFFENSES'
 'PORNOGRAPHY/OBSCENE MAT' 'PROSTITUTION' 'RECOVERED VEHICLE' 'ROBBERY'
```

```
 'RUNAWAY' 'SECONDARY CODES' 'SEX OFFENSES FORCIBLE'
 'SEX OFFENSES NON FORCIBLE' 'STOLEN PROPERTY' 'SUICIDE' 'SUSPICIOUS OCC'
 'TREA' 'TRESPASS' 'VANDALISM' 'VEHICLE THEFT' 'WARRANTS' 'WEAPON LAWS']
```

Cell 8 - Code

Code:

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

print("Training rows:", X_train.shape[0])
print("Validation rows:", X_valid.shape[0])
```

Output:

Text output 1:

```
Training rows: 702439
Validation rows: 175610
```

Cell 9 - Code

Code:

```
cat_features = ["DayOfWeek", "PdDistrict"]
num_features = ["X", "Y", "Year", "Month", "Day", "Hour", "Minute", "IsWeekend"]

TFIDF_MAX_FEATURES = 7000
SVD_COMPONENTS = 250

preprocessor = ColumnTransformer(
    transformers=[
        ("address_tfidf", TfidfVectorizer(max_features=TFIDF_MAX_FEATURES, ngram_range=(1, 2)), "Address"),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_features),
        ("num", StandardScaler(), num_features)
    ],
    remainder="drop"
)

svd = TruncatedSVD(n_components=SVD_COMPONENTS, random_state=42)
svd_scaler = StandardScaler()

print("TF-IDF max features:", TFIDF_MAX_FEATURES)
print("SVD components:", SVD_COMPONENTS)
```

Output:

Text output 1:

```
TF-IDF max features: 7000
SVD components: 250
```

Cell 10 - Code

Code:

```
start = time.time()

X_train_prepared = preprocessor.fit_transform(X_train)
```

```
X_valid_prepared = preprocessor.transform(X_valid)
X_test_prepared = preprocessor.transform(X_test_final)

print("Prepared sparse shapes:")
print("X_train_prepared:", X_train_prepared.shape)
print("X_valid_prepared:", X_valid_prepared.shape)
print("X_test_prepared:", X_test_prepared.shape)

X_train_svd = svd.fit_transform(X_train_prepared)
X_valid_svd = svd.transform(X_valid_prepared)
X_test_svd = svd.transform(X_test_prepared)

X_train_svd = svd_scaler.fit_transform(X_train_svd).astype("float32")
X_valid_svd = svd_scaler.transform(X_valid_svd).astype("float32")
X_test_svd = svd_scaler.transform(X_test_svd).astype("float32")

del X_train_prepared, X_valid_prepared, X_test_prepared
gc.collect()

print("SVD shapes:")
print(X_train_svd.shape, X_valid_svd.shape, X_test_svd.shape)
print("Preprocessing + PCA/SVD time minutes:", round((time.time() - start) / 60, 2))
```

Output:

Text output 1:

```
Prepared sparse shapes:
X_train_prepared: (702439, 7025)
X_valid_prepared: (175610, 7025)
X_test_prepared: (884262, 7025)
SVD shapes:
(702439, 250) (175610, 250) (884262, 250)
Preprocessing + PCA/SVD time minutes: 1.1
```

Cell 11 - Code

Code:

```
results = {}
trained_models = {}

def train_and_evaluate_sklearn(name, model, X_tr, y_tr, X_val, y_val):
    start = time.time()
    print("=" * 80)
    print("Training:", name)

    model.fit(X_tr, y_tr)

    train_pred = model.predict(X_tr)
    valid_pred = model.predict(X_val)

    train_acc = accuracy_score(y_tr, train_pred)
    valid_acc = accuracy_score(y_val, valid_pred)
    train_time = round((time.time() - start) / 60, 2)

    results[name] = {
        "Train Accuracy": train_acc,
        "Validation Accuracy": valid_acc,
        "Training Time Minutes": train_time
    }

    trained_models[name] = model

    print("Train Accuracy:", train_acc)
    print("Validation Accuracy:", valid_acc)
    print("Training Time Minutes:", train_time)
    print(classification_report(y_val, valid_pred, target_names=label_encoder.classes_))

    gc.collect()
    return model
```

Output:

```
No output captured.
```

Cell 12 - Code

Code:

```
logistic_model = LogisticRegression(
    max_iter=300,
    solver="saga",
    C=1.0,
    n_jobs=-1,
    verbose=1
)

logistic_model = train_and_evaluate_sklearn(
    "Logistic Regression + PCA/SVD",
    logistic_model,
    X_train_svd,
    y_train,
    X_valid_svd,
    y_valid
)
```

Output:

Text output 1:

```
================================================================================
Training: Logistic Regression + PCA/SVD
convergence after 87 epochs took 1394 seconds
Train Accuracy: 0.2644642452938974
```

```
Validation Accuracy: 0.2641193553897842
Training Time Minutes: 23.24
                            precision    recall  f1-score   support

                     ARSON       0.00      0.00      0.00       303
                   ASSAULT       0.17      0.10      0.12     15375
                BAD CHECKS       0.00      0.00      0.00        81
                   BRIBERY       0.00      0.00      0.00        58
                  BURGLARY       0.13      0.01      0.02      7351
         DISORDERLY CONDUCT       0.00      0.00      0.00       864
DRIVING UNDER THE INFLUENCE       0.00      0.00      0.00       454
              DRUG/NARCOTIC       0.29      0.37      0.32     10794
                DRUNKENNESS       0.00      0.00      0.00       856
              EMBEZZLEMENT       0.00      0.00      0.00       233
                 EXTORTION       0.00      0.00      0.00        51
           FAMILY OFFENSES       0.00      0.00      0.00        98
     FORGERY/COUNTERFEITING       0.09      0.00      0.00      2122
                     FRAUD       0.00      0.00      0.00      3336
                  GAMBLING       0.00      0.00      0.00        29
                KIDNAPPING       0.00      0.00      0.00       468
             LARCENY/THEFT       0.28      0.72      0.41     34980
               LIQUOR LAWS       0.00      0.00      0.00       381
                 LOITERING       0.00      0.00      0.00       245
            MISSING PERSON       0.41      0.22      0.29      5198
              NON-CRIMINAL       0.19      0.10      0.14     18461
            OTHER OFFENSES       0.24      0.37      0.30     25236
    PORNOGRAPHY/OBSCENE MAT       0.00      0.00      0.00         4
               PROSTITUTION       0.43      0.57      0.49      1497
          RECOVERED VEHICLE       0.00      0.00      0.00       628
                   ROBBERY       0.00      0.00      0.00      4600
                   RUNAWAY       0.41      0.19      0.26       389
           SECONDARY CODES       0.00      0.00      0.00      1997
      SEX OFFENSES FORCIBLE       0.00      0.00      0.00       878
  SEX OFFENSES NON FORCIBLE       0.00      0.00      0.00        29
           STOLEN PROPERTY       0.00      0.00      0.00       908
                   SUICIDE       0.00      0.00      0.00       102
            SUSPICIOUS OCC       0.00      0.00      0.00      6283
                      TREA       0.00      0.00      0.00         1
                 TRESPASS       0.00      0.00      0.00      1465
                VANDALISM       0.18      0.00      0.00      8945
             VEHICLE THEFT       0.23      0.20      0.21     10756
                  WARRANTS       0.23      0.00      0.01      8443
              WEAPON LAWS       0.00      0.00      0.00      1711

                 accuracy                           0.26    175610
                macro avg       0.08      0.07      0.07    175610
             weighted avg       0.20      0.26      0.20    175610
```

Cell 13 - Code

Code:

```
svm_model = SGDClassifier(
    loss="hinge",
    alpha=0.0001,
    max_iter=1000,
    tol=1e-3,
    random_state=42,
    n_jobs=-1
)

svm_model = train_and_evaluate_sklearn(
    "SVM SGDClassifier + PCA/SVD",
    svm_model,
    X_train_svd,
    y_train,
    X_valid_svd,
```

```
    y_valid
)
```

Output:

Text output 1:

```
================================================================================
Training: SVM SGDClassifier + PCA/SVD
Train Accuracy: 0.1779926797914125
Validation Accuracy: 0.17730197596947783
Training Time Minutes: 3.94
                            precision    recall  f1-score   support

                     ARSON       0.00      0.00      0.00       303
                   ASSAULT       0.11      0.10      0.10     15375
                BAD CHECKS       0.00      0.00      0.00        81
                   BRIBERY       0.00      0.00      0.00        58
                  BURGLARY       0.08      0.08      0.08      7351
         DISORDERLY CONDUCT       0.12      0.01      0.01       864
DRIVING UNDER THE INFLUENCE       0.00      0.00      0.00       454
              DRUG/NARCOTIC       0.20      0.24      0.22     10794
                DRUNKENNESS       0.00      0.00      0.00       856
               EMBEZZLEMENT       0.00      0.00      0.00       233
                 EXTORTION       0.00      0.00      0.00        51
            FAMILY OFFENSES       0.00      0.00      0.00        98
     FORGERY/COUNTERFEITING       0.03      0.06      0.04      2122
                     FRAUD       0.03      0.03      0.03      3336
                  GAMBLING       0.00      0.00      0.00        29
                KIDNAPPING       0.00      0.00      0.00       468
             LARCENY/THEFT       0.28      0.41      0.33     34980
                LIQUOR LAWS       0.00      0.00      0.00       381
                 LOITERING       0.00      0.00      0.00       245
            MISSING PERSON       0.16      0.19      0.18      5198
              NON-CRIMINAL       0.13      0.11      0.12     18461
            OTHER OFFENSES       0.21      0.24      0.23     25236
    PORNOGRAPHY/OBSCENE MAT       0.00      0.00      0.00         4
               PROSTITUTION       0.48      0.36      0.41      1497
          RECOVERED VEHICLE       0.00      0.00      0.00       628
                   ROBBERY       0.03      0.02      0.02      4600
                   RUNAWAY       0.71      0.01      0.03       389
            SECONDARY CODES       0.01      0.01      0.01      1997
       SEX OFFENSES FORCIBLE       0.00      0.00      0.00       878
  SEX OFFENSES NON FORCIBLE       0.00      0.00      0.00        29
            STOLEN PROPERTY       0.00      0.00      0.00       908
                   SUICIDE       0.00      0.00      0.00       102
             SUSPICIOUS OCC       0.03      0.03      0.03      6283
                      TREA       0.00      0.00      0.00         1
                  TRESPASS       0.04      0.01      0.02      1465
                 VANDALISM       0.07      0.03      0.05      8945
              VEHICLE THEFT       0.14      0.11      0.12     10756
                  WARRANTS       0.07      0.04      0.06      8443
               WEAPON LAWS       0.02      0.01      0.01      1711

                  accuracy                           0.18    175610
                 macro avg       0.08      0.05      0.05    175610
              weighted avg       0.15      0.18      0.16    175610
```

Cell 14 - Code

Code:

```
decision_tree_model = DecisionTreeClassifier(
    max_depth=40,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
```

```
)

decision_tree_model = train_and_evaluate_sklearn(
    "Decision Tree + PCA/SVD",
    decision_tree_model,
    X_train_svd,
    y_train,
    X_valid_svd,
    y_valid
)
```

Output:

Text output 1:

```
================================================================================
Training: Decision Tree + PCA/SVD
Train Accuracy: 0.5081167190318305
Validation Accuracy: 0.2389271681567109
Training Time Minutes: 14.59
                             precision    recall  f1-score   support

                      ARSON       0.04      0.00      0.01       303
                    ASSAULT       0.16      0.23      0.19     15375
                BAD CHECKS       0.00      0.00      0.00        81
                   BRIBERY       0.00      0.00      0.00        58
                  BURGLARY       0.11      0.12      0.12      7351
         DISORDERLY CONDUCT       0.05      0.02      0.03       864
DRIVING UNDER THE INFLUENCE       0.02      0.00      0.00       454
              DRUG/NARCOTIC       0.29      0.35      0.32     10794
                DRUNKENNESS       0.04      0.01      0.01       856
               EMBEZZLEMENT       0.04      0.00      0.01       233
                  EXTORTION       0.00      0.00      0.00        51
           FAMILY OFFENSES       0.00      0.00      0.00        98
     FORGERY/COUNTERFEITING       0.10      0.06      0.07      2122
                      FRAUD       0.07      0.03      0.04      3336
                   GAMBLING       0.00      0.00      0.00        29
                 KIDNAPPING       0.00      0.00      0.00       468
             LARCENY/THEFT       0.34      0.44      0.38     34980
               LIQUOR LAWS       0.05      0.01      0.01       381
                  LOITERING       0.30      0.14      0.19       245
            MISSING PERSON       0.37      0.35      0.36      5198
              NON-CRIMINAL       0.18      0.21      0.20     18461
            OTHER OFFENSES       0.25      0.28      0.26     25236
    PORNOGRAPHY/OBSCENE MAT       0.00      0.00      0.00         4
               PROSTITUTION       0.50      0.52      0.51      1497
          RECOVERED VEHICLE       0.08      0.01      0.01       628
                    ROBBERY       0.07      0.03      0.04      4600
                    RUNAWAY       0.34      0.18      0.24       389
           SECONDARY CODES       0.04      0.00      0.01      1997
      SEX OFFENSES FORCIBLE       0.07      0.01      0.02       878
  SEX OFFENSES NON FORCIBLE       0.00      0.00      0.00        29
            STOLEN PROPERTY       0.00      0.00      0.00       908
                    SUICIDE       0.50      0.02      0.04       102
             SUSPICIOUS OCC       0.05      0.02      0.03      6283
                       TREA       0.00      0.00      0.00         1
                  TRESPASS       0.06      0.01      0.02      1465
                 VANDALISM       0.10      0.07      0.08      8945
              VEHICLE THEFT       0.27      0.28      0.28     10756
                   WARRANTS       0.11      0.07      0.09      8443
               WEAPON LAWS       0.08      0.02      0.03      1711

                  accuracy                           0.24    175610
                 macro avg       0.12      0.09      0.09    175610
              weighted avg       0.21      0.24      0.22    175610
```

Cell 15 - Code

Code:

```
X_train_cnn = X_train_svd.reshape(X_train_svd.shape[0], X_train_svd.shape[1], 1)
X_valid_cnn = X_valid_svd.reshape(X_valid_svd.shape[0], X_valid_svd.shape[1], 1)
X_test_cnn = X_test_svd.reshape(X_test_svd.shape[0], X_test_svd.shape[1], 1)

y_train_cat = to_categorical(y_train, num_classes=num_classes)
y_valid_cat = to_categorical(y_valid, num_classes=num_classes)

print("CNN train shape:", X_train_cnn.shape)
print("CNN validation shape:", X_valid_cnn.shape)
print("CNN test shape:", X_test_cnn.shape)
```

Output:

Text output 1:

```
CNN train shape: (702439, 250, 1)
CNN validation shape: (175610, 250, 1)
CNN test shape: (884262, 250, 1)
```

Cell 16 - Code

Code:

```
gc.collect()
start = time.time()

cnn_model = Sequential([
    Input(shape=(X_train_cnn.shape[1], 1)),
    Conv1D(32, kernel_size=3, activation="relu"),
    MaxPooling1D(pool_size=2),
    Dropout(0.25),

    Conv1D(64, kernel_size=3, activation="relu"),
    MaxPooling1D(pool_size=2),
    Dropout(0.25),

    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.4),
    Dense(num_classes, activation="softmax")
])

cnn_model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_model.summary()

callbacks = [
    EarlyStopping(monitor="loss", patience=3, restore_best_weights=True),
    ReduceLROnPlateau(monitor="loss", factor=0.5, patience=2, min_lr=1e-6)
]

history = cnn_model.fit(
    X_train_cnn,
    y_train_cat,
    epochs=15,
    batch_size=256,
    callbacks=callbacks,
    verbose=1
)

cnn_train_loss, cnn_train_acc = cnn_model.evaluate(X_train_cnn, y_train_cat, batch_size=256, verbose=0)
cnn_valid_loss, cnn_valid_acc = cnn_model.evaluate(X_valid_cnn, y_valid_cat, batch_size=256, verbose=0)

cnn_time = round((time.time() - start) / 60, 2)

results["CNN + PCA/SVD"] = {
    "Train Accuracy": cnn_train_acc,
    "Validation Accuracy": cnn_valid_acc,
    "Training Time Minutes": cnn_time
}

trained_models["CNN + PCA/SVD"] = cnn_model

print("CNN Train Accuracy:", cnn_train_acc)
print("CNN Validation Accuracy:", cnn_valid_acc)
print("CNN Training Time Minutes:", cnn_time)
```

Output:

Text output 1:

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are
installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

Display output 2:

```
[1mModel: "sequential"[0m
```

Display output 3:

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃[1m [0m[1mLayer (type)                   [0m[1m [0m┃[1m [0m[1mOutput Shape          [0m[1m [0m┃[1m [0m[1m      Param #[0m[1m [0m┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ conv1d ([38;5;33mConv1D[0m)                 │ ([38;5;45mNone[0m, [38;5;34m248[0m, [38;5;34m32[0m)        │ [38;5;34m128[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ max_pooling1d ([38;5;33mMaxPooling1D[0m)    │ ([38;5;45mNone[0m, [38;5;34m124[0m, [38;5;34m32[0m)        │ [38;5;34m0[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ dropout ([38;5;33mDropout[0m)               │ ([38;5;45mNone[0m, [38;5;34m124[0m, [38;5;34m32[0m)        │ [38;5;34m0[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ conv1d_1 ([38;5;33mConv1D[0m)               │ ([38;5;45mNone[0m, [38;5;34m122[0m, [38;5;34m64[0m)        │ [38;5;34m6,208[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ max_pooling1d_1 ([38;5;33mMaxPooling1D[0m)  │ ([38;5;45mNone[0m, [38;5;34m61[0m, [38;5;34m64[0m)         │ [38;5;34m0[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ dropout_1 ([38;5;33mDropout[0m)             │ ([38;5;45mNone[0m, [38;5;34m61[0m, [38;5;34m64[0m)         │ [38;5;34m0[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ flatten ([38;5;33mFlatten[0m)               │ ([38;5;45mNone[0m, [38;5;34m3904[0m)           │             [38;5;34m0[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ dense ([38;5;33mDense[0m)                   │ ([38;5;45mNone[0m, [38;5;34m128[0m)            │       [38;5;34m499,840[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ dropout_2 ([38;5;33mDropout[0m)             │ ([38;5;45mNone[0m, [38;5;34m128[0m)            │             [38;5;34m0[0m │
├─────────────────────────────────┼──────────────────────────────┼───────────────┤
│ dense_1 ([38;5;33mDense[0m)                 │ ([38;5;45mNone[0m, [38;5;34m39[0m)             │         [38;5;34m5,031[0m │
└─────────────────────────────────┴──────────────────────────────┴───────────────┘
```

Display output 4:

```
[1m Total params: [0m[38;5;34m511,207[0m (1.95 MB)
```

Display output 5:

```
[1m Trainable params: [0m[38;5;34m511,207[0m (1.95 MB)
```

Display output 6:

```
[1m Non-trainable params: [0m[38;5;34m0[0m (0.00 B)
```

Text output 7:

```
Epoch 1/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m42s[0m 15ms/step - accuracy: 0.2461 - loss: 2.5377 - learning_rate: 0.0010
Epoch 2/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m43s[0m 16ms/step - accuracy: 0.2601 - loss: 2.4713 - learning_rate: 0.0010
Epoch 3/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m41s[0m 15ms/step - accuracy: 0.2637 - loss: 2.4525 - learning_rate: 0.0010
Epoch 4/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m39s[0m 14ms/step - accuracy: 0.2670 - loss: 2.4402 - learning_rate: 0.0010
Epoch 5/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m40s[0m 14ms/step - accuracy: 0.2679 - loss: 2.4329 - learning_rate: 0.0010
Epoch 6/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m38s[0m 14ms/step - accuracy: 0.2697 - loss: 2.4271 - learning_rate: 0.0010
```

```
Epoch 7/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m39s[0m 14ms/step - accuracy: 0.2705 - loss: 2.4232 - learning_rate: 0.0010
Epoch 8/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m40s[0m 14ms/step - accuracy: 0.2716 - loss: 2.4191 - learning_rate: 0.0010
Epoch 9/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m40s[0m 15ms/step - accuracy: 0.2721 - loss: 2.4157 - learning_rate: 0.0010
Epoch 10/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m44s[0m 16ms/step - accuracy: 0.2728 - loss: 2.4130 - learning_rate: 0.0010
Epoch 11/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m40s[0m 15ms/step - accuracy: 0.2733 - loss: 2.4114 - learning_rate: 0.0010
Epoch 12/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m39s[0m 14ms/step - accuracy: 0.2740 - loss: 2.4097 - learning_rate: 0.0010
Epoch 13/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m40s[0m 15ms/step - accuracy: 0.2743 - loss: 2.4074 - learning_rate: 0.0010
Epoch 14/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m41s[0m 15ms/step - accuracy: 0.2746 - loss: 2.4061 - learning_rate: 0.0010
Epoch 15/15
[1m2744/2744[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m40s[0m 15ms/step - accuracy: 0.2750 - loss: 2.4040 - learning_rate: 0.0010
CNN Train Accuracy: 0.2902771532535553
CNN Validation Accuracy: 0.2872387766838074
CNN Training Time Minutes: 10.32
```

Cell 17 - Code

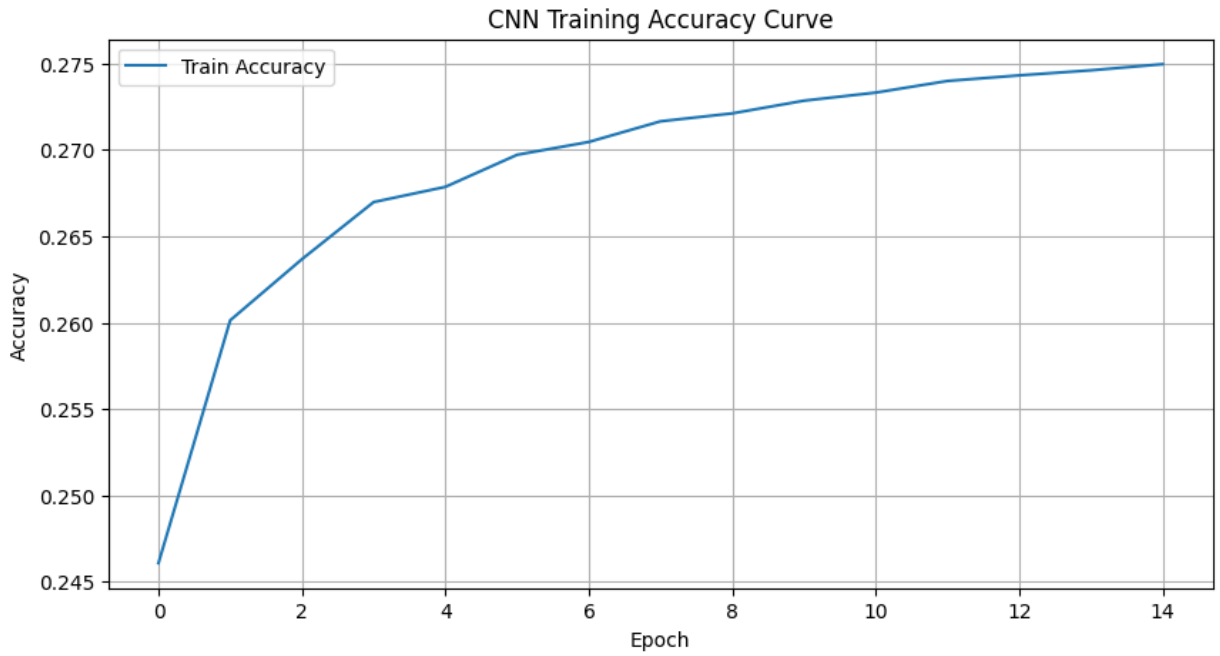
Code:

```
plt.figure(figsize=(10, 5))
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.title("CNN Training Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()

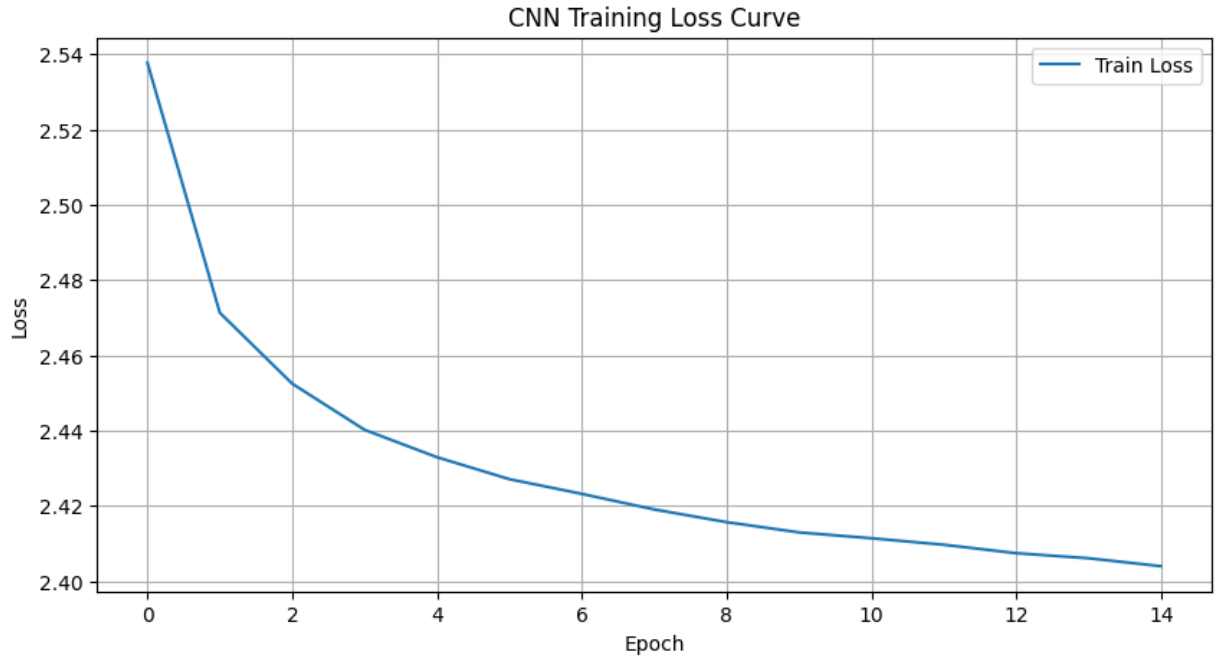
plt.figure(figsize=(10, 5))
plt.plot(history.history["loss"], label="Train Loss")
plt.title("CNN Training Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
```

Output:

Screenshot output 1:



Screenshot output 2:



Cell 18 - Code

Code:

```
results_df = pd.DataFrame(results).T
results_df = results_df.sort_values("Validation Accuracy", ascending=False)
display(results_df)
```

Output:

Display output 1:

```
                            Train Accuracy  Validation Accuracy  \
CNN + PCA/SVD                     0.290277             0.287239
Logistic Regression + PCA/SVD     0.264464             0.264119
```

```
Decision Tree + PCA/SVD              0.508117             0.238927
SVM SGDClassifier + PCA/SVD          0.177993             0.177302

                                Training Time Minutes
CNN + PCA/SVD                                   10.32
Logistic Regression + PCA/SVD                   23.24
Decision Tree + PCA/SVD                         14.59
SVM SGDClassifier + PCA/SVD                      3.94
```

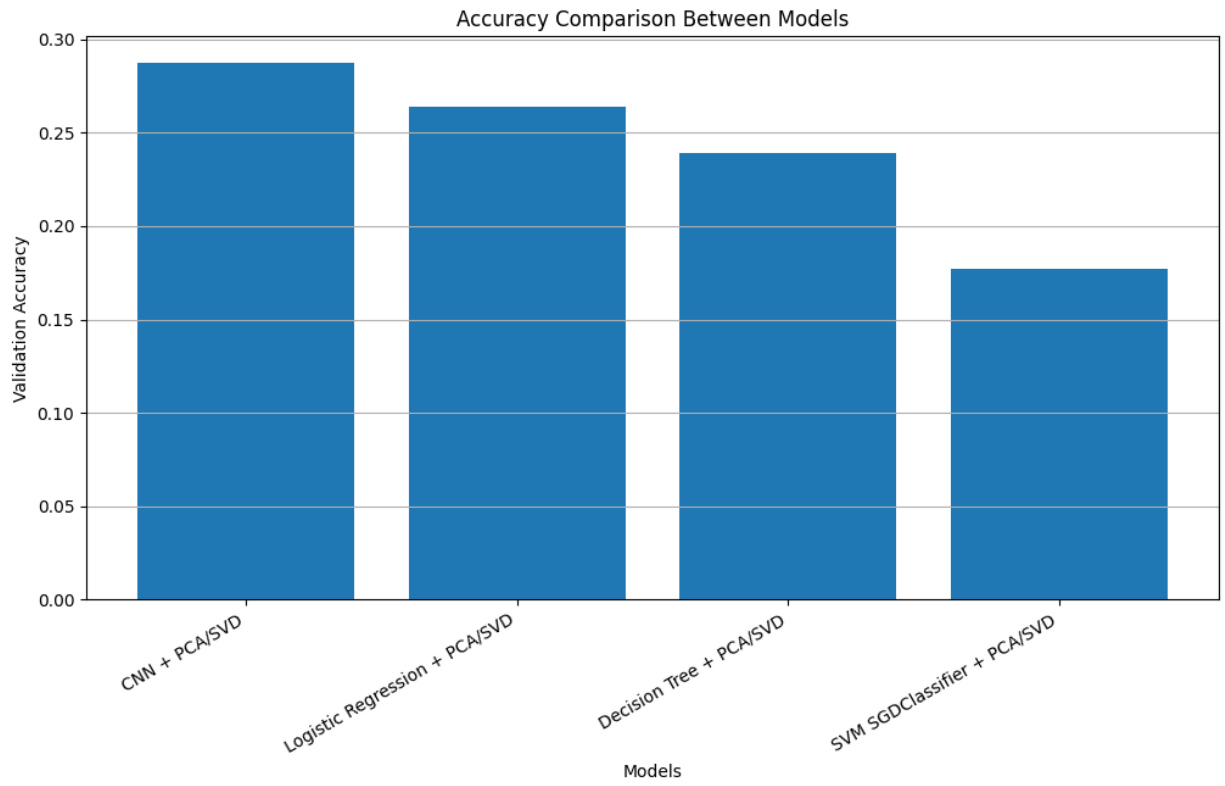
Cell 19 - Code

Code:

```
plt.figure(figsize=(12, 6))
plt.bar(results_df.index, results_df["Validation Accuracy"])
plt.title("Accuracy Comparison Between Models")
plt.xlabel("Models")
plt.ylabel("Validation Accuracy")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y")
plt.show()
```

Output:

Screenshot output 1:



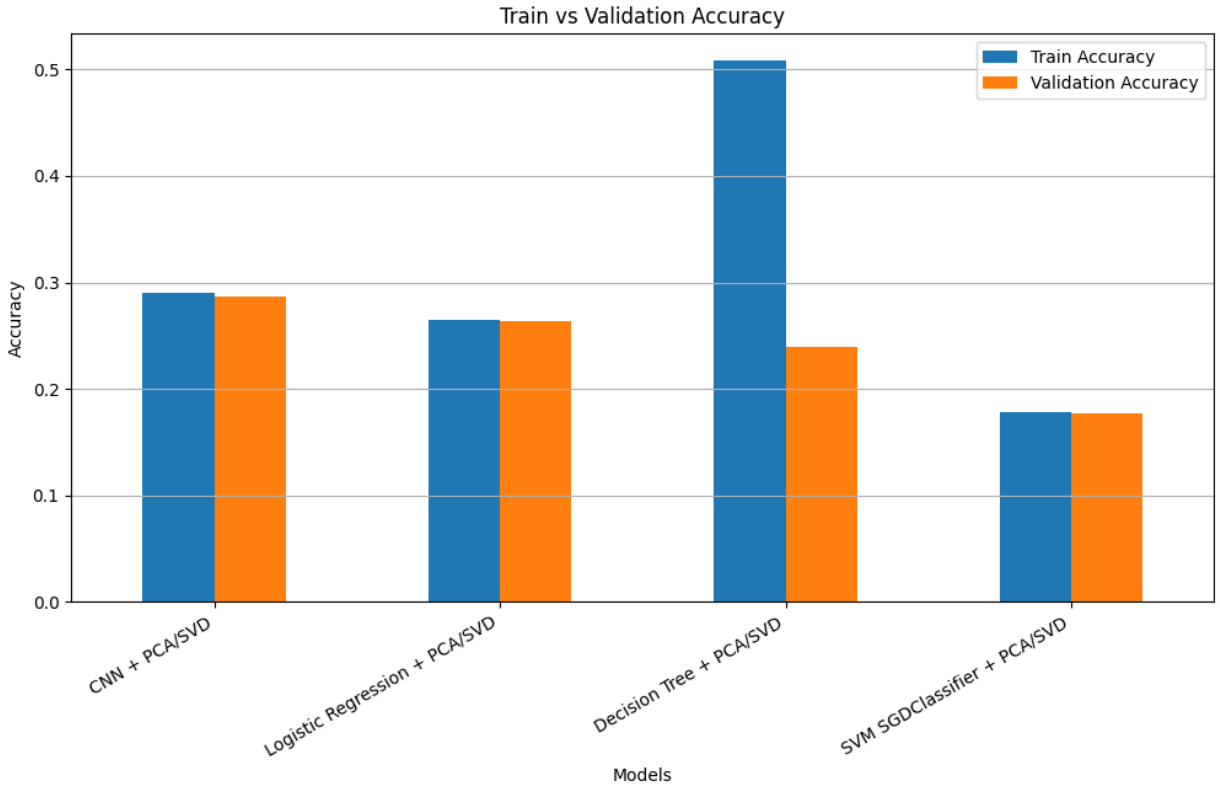
Cell 20 - Code

Code:

```
results_df[["Train Accuracy", "Validation Accuracy"]].plot(kind="bar", figsize=(12, 6))
plt.title("Train vs Validation Accuracy")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y")
plt.show()
```

Output:

Screenshot output 1:



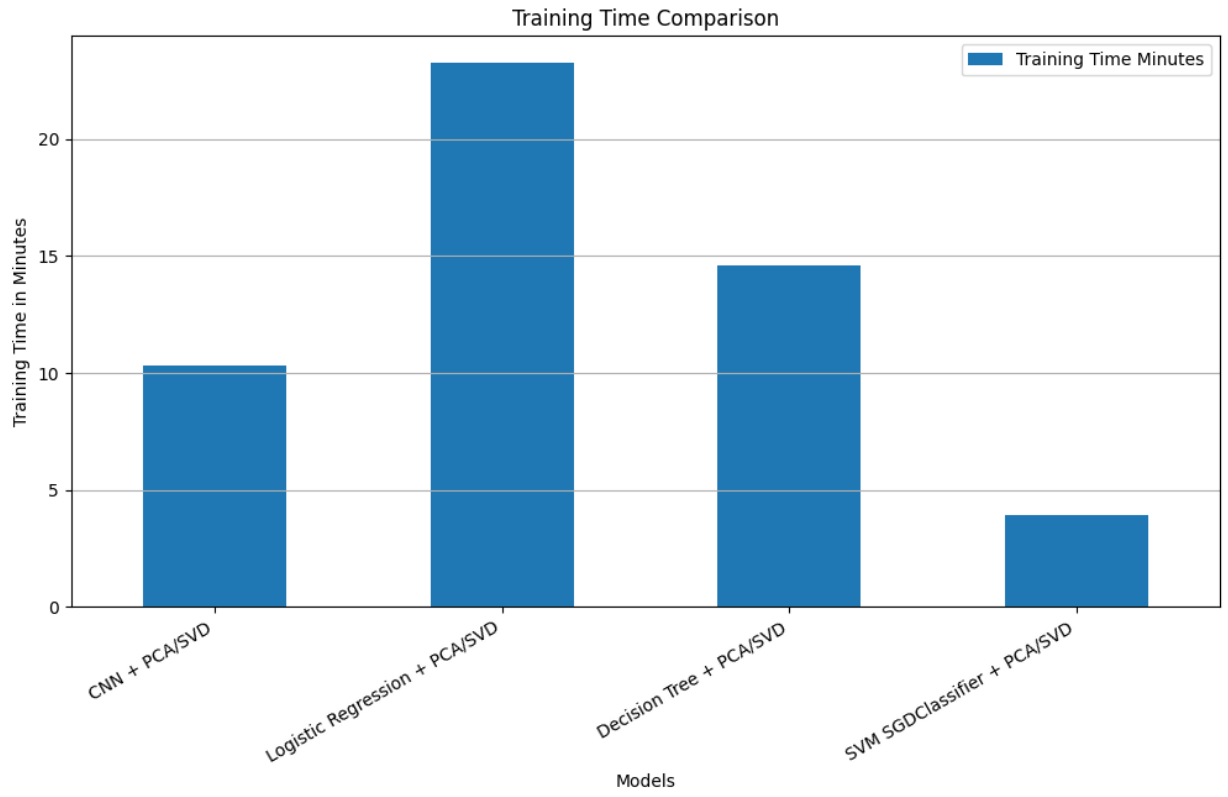
Cell 21 - Code

Code:

```
results_df[["Training Time Minutes"]].plot(kind="bar", figsize=(12, 6))
plt.title("Training Time Comparison")
plt.xlabel("Models")
plt.ylabel("Training Time in Minutes")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y")
plt.show()
```

Output:

Screenshot output 1:



Cell 22 - Code

Code:

```
best_model_name = results_df.index[0]
best_model = trained_models[best_model_name]

print("Best Model:", best_model_name)
print("Best Validation Accuracy:", results_df.iloc[0]["Validation Accuracy"])
```

Output:

Text output 1:

```
Best Model: CNN + PCA/SVD
Best Validation Accuracy: 0.2872387766838074
```

Cell 23 - Code

Code:

```
if best_model_name == "CNN + PCA/SVD":
    y_pred_proba = best_model.predict(X_valid_cnn, batch_size=256)
    y_pred = np.argmax(y_pred_proba, axis=1)
else:
    y_pred = best_model.predict(X_valid_svd)

cm = confusion_matrix(y_valid, y_pred)

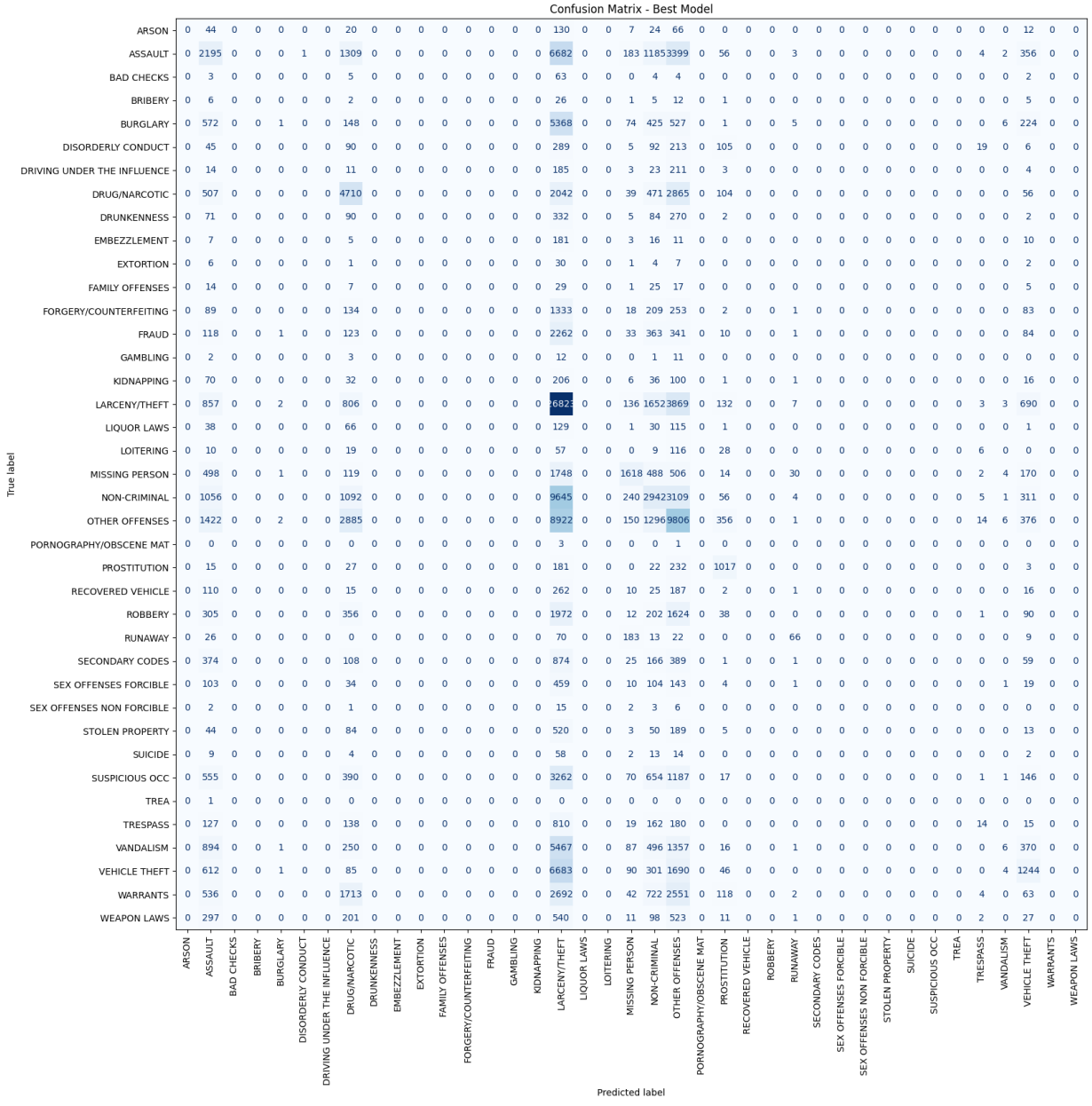
fig, ax = plt.subplots(figsize=(18, 18))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_encoder.classes_)
disp.plot(ax=ax, xticks_rotation=90, cmap="Blues", values_format="d", colorbar=False)
plt.title("Confusion Matrix - Best Model")
plt.show()
```

Output:

Text output 1:

```
[1m686/686[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m3s[0m 4ms/step
```

Screenshot output 2:



Cell 24 - Code

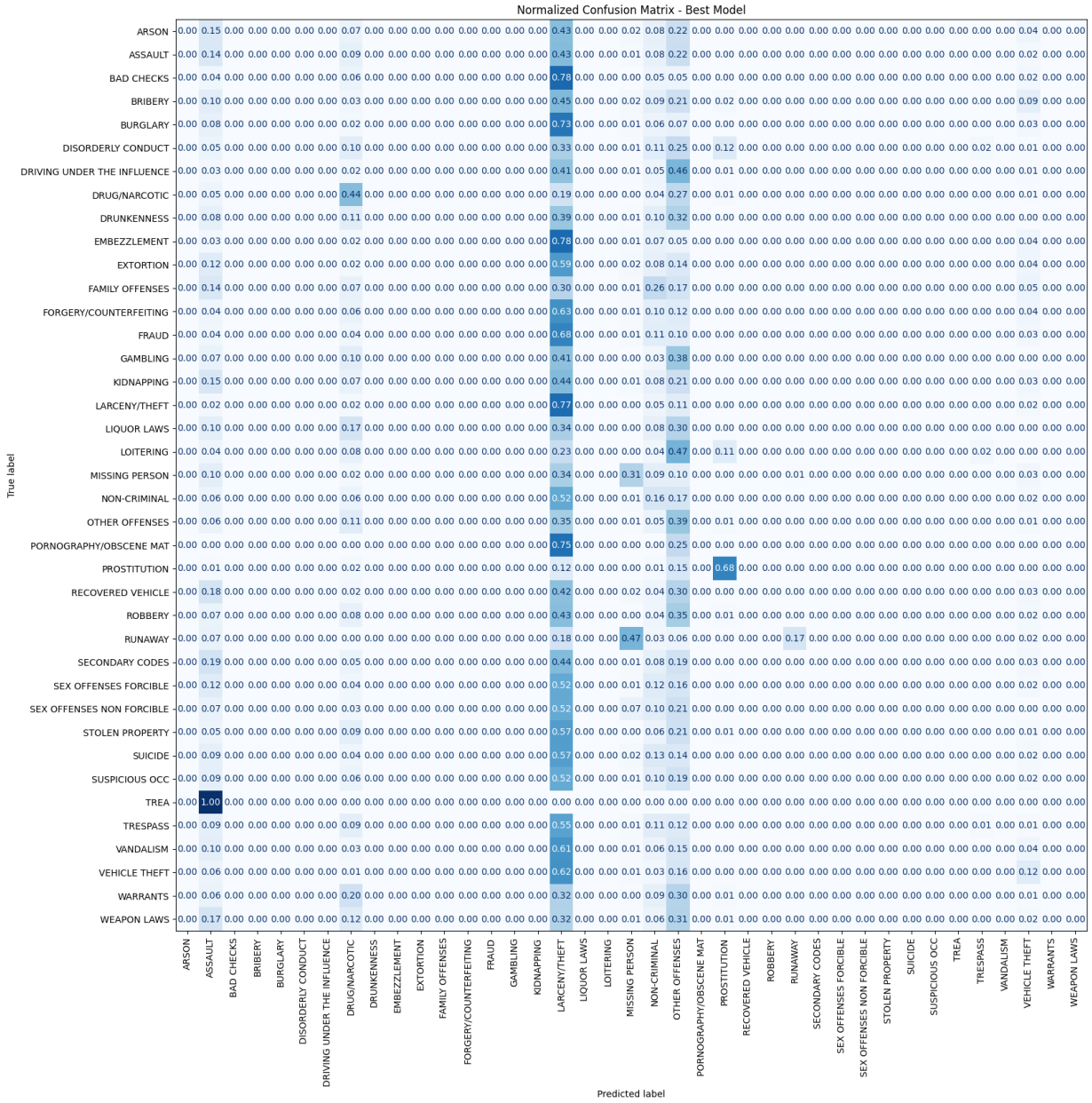
Code:

```
cm_normalized = confusion_matrix(y_valid, y_pred, normalize="true")

fig, ax = plt.subplots(figsize=(18, 18))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_normalized, display_labels=label_encoder.classes_)
disp.plot(ax=ax, xticks_rotation=90, cmap="Blues", values_format=".2f", colorbar=False)
plt.title("Normalized Confusion Matrix - Best Model")
plt.show()
```

Output:

Screenshot output 1:



Cell 25 - Code

Code:

```
joblib.dump(preprocessor, "preprocessor.pkl")
joblib.dump(svd, "svd.pkl")
```

```
joblib.dump(svd_scaler, "svd_scaler.pkl")
joblib.dump(label_encoder, "label_encoder.pkl")

if best_model_name == "CNN + PCA/SVD":
    best_model.save("final_best_model_cnn.keras")
    print("Saved best CNN model: final_best_model_cnn.keras")
else:
    joblib.dump(best_model, "final_best_model.pkl")
    print("Saved best sklearn model: final_best_model.pkl")

print("Preprocessing objects saved successfully.")
```

Output:

Text output 1:

```
Saved best CNN model: final_best_model_cnn.keras
Preprocessing objects saved successfully.
```

Cell 26 - Code

Code:

```
if best_model_name == "CNN + PCA/SVD":
    test_pred_proba = best_model.predict(X_test_cnn, batch_size=256)
    test_pred_encoded = np.argmax(test_pred_proba, axis=1)
else:
    test_pred_encoded = best_model.predict(X_test_svd)

test_pred = label_encoder.inverse_transform(test_pred_encoded)

submission = pd.DataFrame({
    "Id": test_ids,
    "Category": test_pred
})

submission.to_csv("submission.csv", index=False)
display(submission.head())
print("submission.csv saved successfully.")
```

Output:

Text output 1:

```
[1m3455/3455[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m13s[0m 4ms/step
```

Display output 2:

	Id	Category
0	0	OTHER OFFENSES
1	1	OTHER OFFENSES
2	2	LARCENY/THEFT
3	3	OTHER OFFENSES
4	4	OTHER OFFENSES

Text output 3:

```
submission.csv saved successfully.
```

Cell 27 - Markdown

Report Notes

- Full dataset was used without sampling.
- `Descript` and `Resolution` were removed to avoid data leakage.
- TF-IDF was applied to Address.
- OneHotEncoder was applied to categorical features.
- StandardScaler was applied to numerical features.
- TruncatedSVD was used as PCA-like dimensionality reduction.
- Logistic Regression, SVM, Decision Tree, and CNN were implemented.
- Accuracy comparison and confusion matrix were added.

Discussion Preparation Notes

used TF-IDF

Because Address is text and models need numeric input. TF-IDF represents important words numerically.

used TruncatedSVD instead of PCA

Because TF-IDF produces sparse features. TruncatedSVD is memory-efficient for sparse data and works as PCA-like dimensionality reduction.

removed Descript and Resolution

Because They can leak the answer because they contain direct or post-event information about the crime.

SVM implemented with SGDClassifier

Because It uses hinge loss like linear SVM but is scalable for very large datasets.

use confusion matrix

Because It shows which categories are confused with each other and helps analyze detailed errors.